

Modelos y Paradigmas de Programación

Estudio del lenguaje de programación

Lua

Universidad del Valle

[redacted]

19 de diciembre de 2021

Resumen

En la actualidad existen muchos lenguajes de programación. Los cuales nacen para resolver problemas de acuerdo a unas necesidades. Estos lenguajes parten de algo núcleo que son *los modelos y paradigmas de programación*. Los cuales buscan la manera de solucionar dichos problemas. Para esto se le hizo un estudio al lenguaje de programación LUA el cual es un lenguaje de programación script que nos permite realizar automatizaciones de programas escritos en C o C++ como muchas otras cosas mas. Aparte que dicho lenguaje abarca varios paradigmas como lo son el orientado a objetos, funcional, procedimental y orientado a datos. También su estructura de datos tabla la cual es muy poderosa gracias a su expresividad.

1. Introducción

Lua es un poderoso, eficiente, ligero y integrable lenguaje de programación de tipo script. El cual soporta programación procedimental, orientada a objetos, funcional, programación basada en datos y descripción de datos [2].

Lua es un lenguaje de programación, el cual fue diseñado, implementado y mantenido por el grupo de PUC-RIO, Pontificia Universidad Católica de Río de Janeiro. Ahora alojada en los laboratorios de LabLua, un laboratorio del departamento de ciencias de la computación de PUC-Rio[1].

Este lenguaje de programación es de tipado dinámico, el cual corre de manera interpretada en una maquina virtual basada en *registros*. Lo cual la hace ideal para configurar, crear stripts y tambien modelar prototipos.

Lua nace de una implementación limpia de C y C++. El cual lo que permite es tener compatibilidad con programas que estén escritos en estos. Usando su modelo de scripting el cual le permite interactuar con ellos. Lua no tiene el concepto de una clase main. Lo que el hace es trabajar junto al dominio del cliente, el cual es llamado *programacion incrustada* o simplemente dominio.

2. Conceptos Particulares

2.1. Que es un Lenguaje de script ?

Este tipo de lenguaje ejecutan diferentes acciones en un entorno de tiempo de ejecución particular, automatizar acciones de un programa, realizar configuraciones, extraer datos y otras muchas posibilidades mas. Donde a lo que nos referimos como un entorno particular es a programas ya hechos los cuales permitan implementaciones de lenguajes de tipo script.[3]

Un lenguaje de tipo script es un lenguaje de programación, pero un lenguaje de programación no es un lenguaje script no son homogéneos en ese sentido, dado a como se modelan y se ejecutan estos mismos. Lua por ejemplo es un lenguaje de tipo script el cual se ejecuta en entornos de C o C++. Automatizar y mejorar el rendimiento de ciertos programas a través de *scripts*, los cuales son pequeñas instrucciones que se ejecutan según se le programen.

Lua en este aspecto es muy utilizado en lo que vienen siendo los videojuegos, donde lo que permite es automatizar ciertas interacciones repetitivas que se pueda tener en algún juego. Casos particulares son como World of Warcraft tiene implementaciones con dicho lenguaje.

3. Modelo de Programación declarativo

Lamentablemente LUA es un lenguaje que es soportado desde el punto de vista imperativo, donde su estructura no soporta casos como ligadura,variables de flujo o valores parciales. No existen en dicho lenguaje como veremos a continuación.

3.1. Variables

Como se dijo anteriormente Lua es un lenguaje de programación de tipado dinámico. El cual sus variables no tienen tipo pero los valores si *(string, int,float, entre otros)*. Todos los valores son valores de primera clase.[5] Aparte de esto

O sea, qué significa esto?

Lua implementa tres tipos de instanciación de variables global,local y campo tabla. [6] Donde este ultimo se trata de un tipo espacial de variable que puede contener cualquier valor excepto nulo. Lamentablemente en la investigación hecha para este lenguaje no se encontró una definición que nos pudiera afirmar si en dicho lenguaje se puede ligar una variable. Por ende no se soporta el modelo de programación declarativo.

No explícitamente, pero uno podría programar declarativamente, de forma implícita... sin usar, por ejemplo, la asignación para cambiar valores a una variable?



Figura 1: Asignación de Valor a una variable

O sea, la asignación típica... O sea las variables son celdas

Para darle el valor a una variable utilizamos el operador =, el cual no cumple con el significado aritmético de que dos cosas son iguales. En lua dicho signo lo que hace es decirle que *haga que dos cosas sean iguales* y de este modo podemos usar la variable declarada a la izquierda en sustituto de la derecha. Gracias a esto podemos darle valores a variables particulares en cualquier momento como vemos en la figura 1.

Aparte de esto también decimos que las variables en Lua en su mayoría son de paso por valor. Debido a que dicho lenguaje no guarda en si la referencia de la variable. Logrando que si una variable toma el valor de otra, esta se pueda modificar sin afectar a la otra.

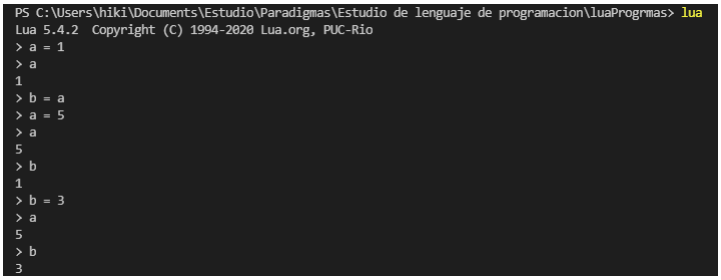


Figura 2: Asignación de valor a una variable, a partir de otra

Sin embargo si utilizamos la estructura de dato tabla cambian las cosas, ya que esta utiliza el paso por referencia debido a que se guarda específicamente en un espacio en memoria donde si le pasamos por valor la tabla a otra variable. Si se modifica en alguna de sus propiedades, entonces se vera reflejado en las dos variables.

Las etiquetas o nombre que le asignamos a la variables en lua pueden ser letras o guión bajo al inicio, luego de cumplir esta podemos agregarle números, pero no pueden empezar con numero. También existen casos especiales

No dijiste nada de las funciones o procedimientos. Hay genericidad? Embebimiento? Instanciación?

```
PS C:\Users\hiki\Documents\Estudio\Paradigmas\Estudio de lenguaje de programacion\luaProgrmas> lua
Lua 5.4.2  Copyright (C) 1994-2020 Lua.org, PUC-Rio
> a = {1,2,3}
> a
table: 00000000007597a0
> a[1]
1
> b=a
> b
table: 00000000007597a0
> b[1]
1
> b[1]= 3
> b[1]
3
> a[1]
3
```

Figura 3: Creación de una tabla, mirando como es paso por referencia

de palabras que ya tienen referencia dentro del lenguaje como por ejemplo else,break,for, entre otros. Tampoco se pueden usar caracteres especiales.Pero no existe un problema en la sintaxis si empieza con mayúscula o no.

```
PS C:\Users\hiki\Documents\Estudio\Paradigmas\Estudio de lenguaje de programacion\luaProgrmas> lua
Lua 5.4.2  Copyright (C) 1994-2020 Lua.org, PUC-Rio
> a =1
> A=2
> a
1
> A
2
> _ = 2
> _
2
> __a= 54
> __a=
>> 1
> __a
1
> a@
stdin:1: syntax error near '@'
>
> a1
nil
```

Figura 4: Declaración de Variables

4. Modelo de programación concurrente declarativo

En Lua se implementa el modelo concurrente, pero lamentablemente no el declarativo. Ya que hace uso imprescindible del concepto de **corrutina** el cual es del modelo de concurrencia por estado y aparte es secuencial.Esto se debe a que no tenemos implementado las variables de flujo, ni los valores parciales.

Las corrutinas son la manera de manejar concurrencia en Lua, estos son un tipo espacial de hilos que implementa este lenguaje el cual es mucho mas sencillo de manejar que los hilos tradicionales. Las corrutinas tienen tres funciones create, resume y yield. Donde la función create recibe una función inicial la cual es la que ejecuta y con la que se crea la corrutina dentro de un hilo, recordando que lua todos sus valores son de primera clase. La funcion resume lo que hace es reiniciar la ejecución de la corrutina que le pasamos. Finalmente tenemos la

función yield que lo que hace es parar o suspender la ejecución de la corrutina y los parámetros que se le pasan son los que se retornan al final de la ejecución.

```lua
corrutina = coroutine.create(function (valor1 , valor2)
    local valorTemporal = 0
    print("Primer llamado a la corrutina", valor1,valor2,valorTemporal)
    valorYild1 = coroutine.yield(valor1+1,valorTemporal)
    valorTemporal = valorTemporal + 1
    print("Segundo llamado a la corrutina", valor1,valor2,valorTemporal)
    valorYield1,valorYield2 =coroutine.yield(valorTemporal, valor2)
...
    return valor1, "end"
...
...

end)


print("main", coroutine.resume(corrutina,1,2))
print("main", coroutine.resume(corrutina,3,6))
```

Figura 5: Creación de una Corrutina

```
PS C:\Users\hiki\Documents\Estudio\Paradigmas\Estudio de lenguaje de programacion\luaProgrmas> lua .\variables.lua
Primer llamado a la corrutina   1       2       0
main    true    2       0
Segundo llamado a la corrutina  1       2       1
main    true    1       2
PS C:\Users\hiki\Documents\Estudio\Paradigmas\Estudio de lenguaje de programacion\luaProgrmas>
```

Figura 6: Ejecución de la Corrutina

Qué es lo que corre concurrentemente? En tu relato, todo sucede secuencialmente...

La figura 5 es un ejemplo de como funcionan las corrutinas en lua. Donde tenemos una corrutina la cual tiene dos estados internos de parada y dependiendo de cuando se llama a esta se ejecuta secuencialmente. En el ejemplo sencillamente lo que pasa en el primer llamado es que se imprime en pantalla los valores que pasamos como parámetros a la corrutina y aparte creamos una variable local inicializada en cero que también se imprime. Luego de que esta imprima se ejecuta la función yield que suspende la corrutina hasta que se vuelva a llamar. En el momento en el que se vuelva a llamar a la corrutina *valorYield1* toma el valor de el primer parámetro que se le pasa a la corrutina cuando se vuelva a llamar, esto es porque la ejecución continua después de el primer yield que tenemos dentro de la corrutina. Aparte de esto como se menciono anteriormente yield es en si el retorno de la función de la corrutina. Por ende, lo que se retorna al final es el valor1 sumándole 1 retornando dos. Se vuelve a llamar a la corrutina se le pasan esta ves como parámetros 3 y 6, mas no va suceder lo mismo que en la primera llamada. En este segundo llamado debemos tener en cuenta que los valores de **valor1 y valor2 no cambiaron** ya que el que recibe los nuevos valores es nuestra variable valorYield1 y solamente toma el primero, ya que asi lo decidimos (si utilizamos múltiple instanciación de variables podemos tomar el otro parámetro que se le pasa a la corrutina). Teniendo ya esto claro lo que sucede a continuación son las instrucciones abajo de valorYield1, que para esta caso lo único que se hace es sumarle uno al valorTermporal y nuevamente imprimir los valores de valor1,valor2 y valorTemporal. En este caso el valorTemporal se va imprimir como 1 y de resto todo seria igual a al primer llamado que se hizo, ya que no utilizamos para nada el parámetro que guardamos en el

valorYield1. Una de las diferencias mas notables de la concurrencia declarativa es el uso de una variable de flujo la cual no tenemos en el modelo concurrente con estado. Aparte de que en la concurrencia declarativa se suspende el hilo y este espera hasta que lo vuelven a llamar. En este caso no es tan simple ya que depende de los estados que tiene implementado que son los que dicen cuando llamar a quien o cuando terminar la ejecución.

Pero hay alguna diferencia con las celdas?

5. Modelo de programación con estados

Como se menciono en el modelo declarativo Lua es un lenguaje donde sus variables y sus valores no son ligados. Sino que se pueden volver a instanciar en otra parte del código. Si comparamos esto con el modelo con estados, podemos percibir que lo mas semejante serian las celdas, ya que son un tipo de variable que están en el almacén mutable y por ende tienen la característica de poder cambiar su valor. Logrando ser natural en la modularidad y la reutilización ya que podemos cambiar los valores de las variables.

6. Modelo de programación concurrente por paso de mensajes

Como se dijo anteriormente Lua para manejar la concurrencia se maneja es a través del modelo de concurrencia por estados y secuencialmente. Donde no tenemos para el caso del modelo concurrente por paso de mensajes. Es notable porque no tenemos la posibilidad de manejar un no determinismo.

7. Modelo de programación orientada a objetos

Uno de los modelos mas conocidos y que mas se utilizan en el mercado es el modelo orientado a objetos. Como se menciono anteriormente Lua también tiene una implementación a objetos, pero a diferencia de los otros lenguajes los cuales utilizan clases como tal para la definición de objetos. Lua usa su estructura de datos tablas, la cual permite implementar funciones de primera clase logrando así poder extender las tablas para poder ser capaz de soportar este modelo. Donde lo que se hace es relacionar las funciones y propiedades que le queramos dar al objeto. También podemos hacer una implementación de una de las cosas mas interesantes de la programación orientada a objetos y esta es la herencia la cual se puede simular a través de meta tablas. Logrando implementar un mecanismo de funciones (métodos) y campos que no existen en los objetos padres.[7].

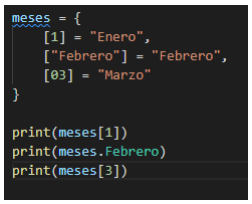


Figura 7: Instanciación de una tabla como un arreglo asociativo

7.1. Tablas

Antes de entrar a validar como es el modelo orientado a objetos miremos primero como es una tabla y de que se trata. Las tablas en lua son el único tipo de estructura valido. El cual permite implementar funciones de primera clase. Donde esta estructura es la que se usa para realizar arreglos como también simular diccionarios, donde este ultimo tiene una similitud muy parecida a nivel de implementación. Las tablas en lua usan arreglos asociativo, el cual lo que nos quiere decir es que un valor va tener asociado un identificador dentro de nuestra estructura de datos tabla.

Si se conoce lo que vienen siendo los diccionarios en lenguajes como lo son python o dart. Podemos ver cierta similitud, solo que no tienen tanta expresividad al darle nombre a las etiquetas y muy probablemente no puedan guardar o almacenar todos las funciones de primera clase como en Lua. También como miramos el concepto de variables, entendimos que las tablas tienen un espacio en memoria determinado, el cual si modificamos alguno de sus elementos internos sucede en este mismo espacio en memoria. Donde si le asignamos dicha tabla a otra variable, al realizar un cambio este se vera reflejado en todas las variables que compartan ese espacio en memoria de la tabla.

7.1.1. MetaTablas

Una meta tabla no es mas que otra tabla que esta relacionada con otra y mediante esta asociación es posible modificar el comportamiento de la tabla a la que esta asociada mediante el uso de meta datos relacionados.

Para hacer esta implementación de meta table utilizamos lo que es la función **setmetatable** la cual nos permite definir cual va ser la tabla principal y cual la meta tabla. Aparte de tener también una función **getmetatable** que nos devuelve los valores de la meta tabla como un constructor.

```lua
mytable = setmetatable({key1 = "value1"}, {
    __index = function(mytable, key)

        if key == "key2" then
            return "metatablevalue"
        else
            return mytable[key]
        end
    end
})

print(mytable.key1,mytable.key2)
```

Figura 8: Implementación de una metatabla

Teniendo ya definido lo que son tablas y meta tablas ahora si podemos hablar de como se implementan estas en el modelo orientado a objetos.

```lua
Rectangle = {area = 0, length = 0, breadth = 0}

function Rectangle:new (o,length,breadth)
    o = o or {}
    setmetatable(o, self)
    self.__index = self
    self.length = length or 0
    self.breadth = breadth or 0
    self.area = length*breadth;
    return o
end

function Rectangle:printArea ()
    print("El Area del rectangulo es ",self.area)
end

r = Rectangle:new(nil,10,20)
r:printArea()
```

Figura 9: Implementación de una clase en Lua

Cómo sabe LUA que new es el constructor?

En el anterior código se muestra una de las características que tiene lua con su estructura de datos Tabla. La cual lo que nos permitió es extender dicha estructura para hacer una implementación de una clase.

Se a dividido de la siguiente manera primero se crea una tabla sencilla en la cual van a estar los atributos base que va tener la clase o objeto, en este caso la clase que vamos a crear es rectángulo. Luego de esto lo que se hace es añadirle una nueva función, la cual va ser el constructor del objeto. Aparte de esto encontramos una nueva sintaxis dentro de lua y es como se implementan métodos a través de funciones, esto se hace con el operador *":"* el cual lo que nos permite es tener un argumento extra conocido como self. Cuando definimos un método de una tabla, lo que logramos con self es acceder a los campos que tiene esta ya definida previamente, es decir, los atributos. Otro método que se crea es el de printArea, el cual lo que hace es retornar un print de el área del rectángulo. Para poder declarar una clase lo que hacemos es crear una variable

y pasarle como valor nuestra clase usando nuestro constructor new, por donde pasamos los atributos que previamente debe de tener. Luego de esto ya quedaría nuestra estructura.

7.2. Herencia

Ahora miremos como se aplica herencia en Lua usando lo que son las meta tablas. Para esto seguiremos usando la clase que teníamos previamente creada.

```lua
Shape = {area = 0}

function Shape:new (o,length,breadth)
   o = o or {}
   setmetatable(o, self)
   self.__index= self
   side = side or 0
   breadth = breadth or 0
   length = length or 0
   self.area = length * breadth
   return o
end


Rectangle = Shape:new()

function Rectangle:new (o,length,breadth)
   o = o or Shape:new(o,length,breadth)
   setmetatable(o, self)
   self.__index = self
   self.length = length or 0
   self.breadth = breadth or 0
   return o
end
function Rectangle:printArea()
   print("Area del Rectangulo", self.area)


end

r = Rectangle:new(nil,10,20)
r:printArea()
```

Figura 10: Implementación de herencia en Lua

Lo que hicimos es crear una nueva tabla la cual se llama Shape (forma) en donde va recibir las medidas del rectángulo y va ser la encargada de sacar y guardar el área del rectángulo específicamente *(no es lo mas adecuado pero se logra entender la idea)*. Si observamos ahora si estamos utilizando lo que es la relación de tablas conocida como meta-tablas la cual definimos con la función *setmetatable* donde se define tanto en la clase forma como en la clase rectángulo para que estas puedan compartir sus atributos y métodos. La forma de decir que

rectángulo hereda de forma es empezar nuestra nueva clase Rectángulo diciendo que es una tabla de tipo forma la cual declaramos vacía. Luego de esto ya lo que hacemos es crear nuestro constructor new donde le asignamos una meta tabla de tipo forma. Esto se hace porque es en dicha meta tabla donde se le asignan los valores.

Como podemos observar la implementación del modelo orientado a objetos en lua es particular en comparación con otros lenguajes de programación, ya que en este se basa principalmente en su estructura de datos tabla para implementar dicho paradigma. Lográndolo de cierta manera. Pero si lo miramos desde la perspectiva del modelo es un poco mas compleja y mas cerrada a la hora de que

Cómo se decide cuál método es el que se invoca? Los atributos de Shape y de Rectángulo son diferentes o son los mismos? Cada que se crea un rectángulo se crean nuevos atributos? Dónde quedan los métodos guardados? La herencia como tal es sencilla o múltiple?

Qué operaciones hay definidas sobre las tablas? Puedo mezclar relaciones? Puedo componerlas?

hacemos herencia.

8. Modelo de programación relacional

Lamentablemente Lua no tiene hasta este grado de expresividad mas que nada porque no tiene implementado el no determinismo. El cual es el principio fundamental en el modelo relacional. Lua en este sentido si tiene implementado el concepto de relaciones en lo que viene siendo sus tablas que es a través de sus etiquetas y llaves que relacionan y se pueden llamar a un valor de cuerdo a esto. Pero no se compara al modelo relacional.

9. Conclusiones

- **Lua es un lenguaje de tipo script**

  Aunque lua sea un lenguaje de programación tenemos que tener en cuenta que su fuerte y por lo que fue creado es para darle mas peso a su funcionalidad de este tipo. Que es la automatización de procesos en otros programas escritos en C/C++.

- **Lua soporta varios modelos de programación, pero no de tipo declarativo**

  Lua es un lenguaje multi paradigma en el cual encontramos el modelo funcional, procedimental , orientado a objetos y orientada a datos. Pero lamentablemente todo esto lo hace desde el modelo imperativo diciendo entonces que no abarca en muchos aspectos los modelos vistos desde el modelo declarativo.

- **La expresividad de Lua mediante su estructura de datos tabla**

  Una de las cosas mas llamativas de Lua es su estructura de datos tabla, la cual nos permite en el caso de los modelos. Implementar lo que es el orientado a objetos usando las operaciones de meta-tablas y lograr herencia a través de ellas.

Me parece que faltó profundizar sobre este tema que consideras esencial

Referencias

[1] Lua, About. *Where does Lua come from?*, `https://www.lua.org/about.html`.

[2] Lua, About. *What is Lua?*, https://www.lua.org/about.html.

[3] Los 13 principales Lenguajes de Scripts a los que Deberías Prestar Atención en 2021 . *¿Qué Son los Lenguajes de Scripts?*, https://kinsta.com/es/blog/lenguajes-script/.

[4] Lua 5.1 Reference Manual. *2.2 – Values and Types*, https://www.lua.org/manual/5.1/manual.html#2.1.

[5] Lua 5.1 Reference Manual. *2.2 – Values and Types*, https://www.lua.org/manual/5.1/manual.html#2.1.

[6] Lua - Variables. https://www.tutorialspoint.com/lua/lua_variables.htm.

[7] LEARN LUA. *Lua - Object Oriented*, https://www.tutorialspoint.com/lua/lua_object_oriented.htm.